Introduction

Some LeetCode problems test whether you know a trick. Others test whether you can combine three different tricks and notice which constraint tells you which one to use. LeetCode 3505, "Minimum Operations to Make Elements Within K Subarrays Equal," is the second kind, and that's exactly why it shows up in hard-tier interview prep.

The problem looks intimidating at first: non-overlapping windows, equalization costs, and a count constraint all stacked on top of each other. But underneath, it's built from three ideas that show up constantly in interviews — the median minimizes absolute deviation, prefix sums turn repeated summation into O(1) lookups, and dynamic programming turns combinatorial explosion into reused subproblems. If you can see why each piece is needed here, you can recognize the same shape in a dozen other problems that don't look anything like this one on the surface.

Problem Overview

You're given an integer array `nums`, along with two integers `x` and `k`.

You're allowed to increment or decrement any single element by 1, as many times as you like. Each such increment or decrement counts as one operation.

Your goal: make the array contain at least `k` non-overlapping subarrays, each of length exactly `x`, where every element inside a chosen subarray is equal to the same value. Different subarrays can target different values. You want the minimum total number of operations across all chosen windows.

In short — pick `k` disjoint fixed-length windows, flatten each one to a constant value as cheaply as possible, minimize the total cost.

Example

Take `nums = [5, -2, 1, 3, 7, 3, 6, 4, -1]`, with `x = 3` and `k = 2`.

- Window at indices `[1, 3]` → values `-2, 1, 3`. Push everything to `1`: costs `3 + 0 + 2 = 5` ... but the video's walkthrough levels this window to `1` more cheaply by choosing the median, giving `3` operations total for that window.

- Window at indices `[5, 7]` → values `3, 6, 4`. Pushing to the median `4` costs `1 + 2 + 0 = 3` … totaling `5`.

Picking these two particular windows, after computing true median costs correctly, gives a combined total of **8 operations** — which matches what the optimal DP returns for `dp[9][2]`. Each window's cost comes from how far its elements sit from that window's own median, and the two windows don't overlap, which satisfies the problem's requirement.

The output isn't about finding *a* valid pair of windows — it's about proving no other choice of 2 non-overlapping length-3 windows does better.

Intuition

Break the problem into two layers, because that's what it actually is.

Layer one: pricing a single window. If you fix one window and ask "what's the cheapest way to make every element in it equal," you're really asking for the point that minimizes the sum of absolute distances to a fixed set of numbers. That point is the median, not the mean. The mean minimizes squared error (useful for variance, least-squares regression), but operations here cost a flat `1` per unit of distance — that's an absolute-value cost, and absolute-value cost is uniquely minimized at the median. This is a fact worth memorizing, because it resurfaces anywhere "cost equals total distance" shows up.

Layer two: picking which windows. Once you can price any individual window in roughly constant extra work per shift, the outer problem becomes a selection problem: choose `k` non-overlapping windows minimizing total price. This is the classic "weighted interval scheduling" shape — and the signal that DP is the right tool is the small bound on `k` (at most 15). A bound that small screams "make it a dimension of your table," not "something to brute-force over."

The key insight tying both layers together: don't let the outer problem's complexity (weighted selection, non-overlap) leak into the inner problem. Each window is priced in isolation, with equal weight on every element inside it. The median trick works at the window level regardless of what the outer DP is doing.

Brute Force Solution

Idea: Try every combination of `k` non-overlapping windows, and for each window try every possible target value, summing operations directly.

Algorithm: 1. Enumerate all valid sets of `k` disjoint windows of length `x`. 2. For each window in a set, try every candidate value (or at least every value present in the window) and compute total distance. 3. Track the minimum total cost across all window sets.

Advantages: Conceptually simple, no clever math required, easy to verify correctness on tiny inputs.

Disadvantages: Combinatorially explosive. The number of ways to choose `k` non-overlapping windows out of `n` positions grows extremely fast, and re-summing distances for every candidate target per window is wasteful on top of that.

Time complexity: Exponential in the number of window choices, with an added polynomial factor per window for target search — effectively unusable once `n` approaches even a few hundred, let alone 100,000.

Space complexity: Depends on implementation, but typically O(n) per recursive path, with exponential branching.

Optimal Solution

The optimal approach is two independent pieces glued together:

Step 1 — Price every window in O(1) amortized. Maintain a sorted view of the current window's elements as you slide across the array. Because the cost of equalizing a window to its median only depends on the sum of elements above and below the median, a running prefix sum over the sorted window lets you compute that cost with two subtractions — no need to re-sum the whole window every time.

Step 2 — DP over position and windows used. Build a table `dp[i][j]` = minimum cost to place `j` windows using the first `i` elements of the array.

State	Meaning
`dp[i][j]`	Best cost using exactly `j` windows ending by position `i`
Transition 1	Skip position `i`: `dp[i][j] = dp[i-1][j]`
Transition 2	Close a window ending at `i`: `dp[i][j] = min(dp[i][j], dp[i-x][j-1] + cost(window ending at i))`

Because `k ≤ 15`, this table never gets wide — it's `n` rows by at most 16 columns, which keeps the whole DP close to linear in `n`.

Python Code

```
from bisect import insort
from itertools import accumulate


class Solution:
    def minOperations(self, nums: list[int], x: int, k: int) -> int:
        n = len(nums)
        costs = self._window_costs(nums, x)

        INF = float("inf")
        dp = [[INF] * (k + 1) for _ in range(n + 1)]
        for i in range(n + 1):
            dp[i][0] = 0

        for i in range(1, n + 1):
            for j in range(1, k + 1):
                dp[i][j] = dp[i - 1][j]  # skip this position
                if i >= x and dp[i - x][j - 1] != INF:
                    window_cost = costs[i - x]
                    candidate = dp[i - x][j - 1] + window_cost
                    if candidate < dp[i][j]:
                        dp[i][j] = candidate

        return dp[n][k]

    def _window_costs(self, nums: list[int], x: int) -> list[int]:
        n = len(nums)
        window = sorted(nums[:x])
        costs = [self._median_cost(window)]

        for i in range(x, n):
            window.remove(nums[i - x])
            insort(window, nums[i])
            costs.append(self._median_cost(window))

        return costs

    def _median_cost(self, sorted_window: list[int]) -> int:
        mid = len(sorted_window) // 2
        target = sorted_window[mid]
        prefix = list(accumulate(sorted_window))
        total = prefix[-1]

        left_sum = prefix[mid] - sorted_window[mid]  # sum left of median
        left_count = mid
        right_sum = total - prefix[mid]               # sum right of median
        right_count = len(sorted_window) - mid - 1

        left_cost = target * left_count - left_sum
        right_cost = right_sum - target * right_count
        return left_cost + right_cost
```

Code Walkthrough

- **Imports:** `insort` keeps a sliding window sorted in O(x) per shift instead of re-sorting from scratch; `accumulate` builds the prefix sum used by the median-cost formula.
- **`minOperations`:** computes `n`, precomputes the cost of every window starting position via `_window_costs`, then fills the `dp` table.
- **`dp` initialization:** `dp[i][0] = 0` for all `i` — using zero windows always costs zero, no matter how far along you are.
- **Main loop:** for each position `i` and window count `j`, the default is "skip" (carry forward `dp[i-1][j]`). If a window of length `x` can end exactly at `i`, compare closing it against the current best.
- **`_window_costs`:** sorts the first window, then slides one step at a time, removing the outgoing element and inserting the incoming one in sorted order.
- **`_median_cost`:** finds the middle index, treats that value as the target, and uses the prefix sum to compute total distance from everything left and right of the median without re-scanning the window.

Dry Run

Using `nums = [5, -2, 1, 3, 7, 3, 6, 4, -1]`, `x = 3`, `k = 2`:

1. `_window_costs` prices every length-3 window: `[-2,1,5]→cost`, `[-2,1,3]→cost`, `[1,3,7]→cost`, `[3,3,7]→cost`, `[3,6,7]→cost`, `[3,4,6]→cost`, `[-1,4,6]→cost`.
2. The DP fills row by row. By the time `i = 9`, `j = 2`, the table has already considered every pair of non-overlapping windows implicitly, because each cell only depends on a strictly earlier cell plus one new window's cost.
3. `dp[9][2]` comes back as `8`, matching the manually chosen windows at indices `[1,3]` and `[5,7]`.

Complexity Analysis

- **Pricing windows:** O(n · x) in the worst case, since each slide removes and inserts into a sorted Python list, which costs O(x) per operation.
- **Filling the DP table:** O(n · k), and since `k ≤ 15`, this is effectively linear in `n`.
- **Overall:** roughly O(n · x + n · k), which comfortably handles `n = 100,000`.
- **Space:** O(n · k) for the DP table, O(x) for the sliding window.

This is correct because the DP only ever reuses previously computed optimal subproblems — no window's cost is recomputed, and no combination of windows is enumerated explicitly.

Alternative Solutions

A segment-tree or balanced-BST-backed sliding window (instead of `bisect.insort` on a list) can bring the window-pricing step down to O(n log x), which matters more as `x` grows large relative to `n`. It's a reasonable upgrade if the `O(n · x)` term becomes the bottleneck, but for competitive-programming-scale inputs the list-based approach already finishes well within time limits — the video's benchmark shows the optimal version running in microseconds.

Edge Cases

- `k * x > n`: no valid arrangement exists; the DP should naturally leave `dp[n][k]` at infinity if implemented directly (the problem's constraints typically guarantee feasibility, but defensive code should check this).
- All elements identical: every window costs 0, so the answer is 0.
- `x = 1`: every window cost is always 0 (a single element is already "equal to itself"), so the answer should be 0 for any valid `k`.
- Negative numbers: operations only depend on distance, so sign doesn't matter — don't accidentally special-case negative values.
- Duplicate values across windows: doesn't affect correctness, since each window is priced independently.

Common Mistakes

1. Using the mean instead of the median to price a window — minimizes the wrong error metric.
2. Recomputing the full sum of distances for every window from scratch instead of using a prefix sum, blowing the time budget.
3. Forgetting that windows must be non-overlapping, and allowing the DP transition to double-count positions.
4. Off-by-one errors in the window-ending transition (`dp[i-x][j-1]` vs. `dp[i-x+1][j-1]`).
5. Not initializing `dp[i][0] = 0` for all `i`, which silently breaks every subsequent transition.
6. Treating `k` as if it could be large and defaulting to a slower algorithm, missing the hint in the constraints.

Interview Questions

- Why does the median minimize sum of absolute differences, and how would you prove it?
- How would the solution change if windows were allowed to overlap?
- What happens to time complexity if `x` can be as large as `n`?
- Could you solve the window-pricing step with a different data structure for better complexity?
- How would you extend this to variable-length windows?

Similar Problems

- **LeetCode 462 (Minimum Moves to Equal Array Elements II):** same median-minimizes-cost principle, applied to the whole array instead of a window.
- **LeetCode 2009 (Minimum Number of Operations to Make Array Continuous):** another DP-plus-window-pricing hybrid.
- **LeetCode 1335 (Minimum Difficulty of a Job Schedule):** same "partition into groups, DP over position and count" structure.
- **LeetCode 1130 (Minimum Cost Tree From Leaf Values):** different cost function, same interval-DP flavor.

Key Takeaways

Median beats mean whenever the cost is a sum of absolute distances. Prefix sums turn repeated range summation into O(1) lookups. And when one input variable is tightly bounded while another is large, that bounded variable almost always belongs as a dimension in a DP table rather than something you brute-force over. Stack these three ideas and a problem that looks exponential collapses into something that runs in microseconds.

Watch the Video

For the full walkthrough — including the cell-by-cell DP trace and the live benchmark comparing brute force against the optimal solution — watch the video here: {LINK}

About the Series

This breakdown is part of *Fun with Learning Technology*, a series dedicated to making hard algorithmic concepts click — not by memorizing patterns, but by understanding why each technique

exists and when to reach for it. Every episode pairs a real problem with the reasoning an experienced engineer would use to solve it live.

Call To Action

If this helped the DP finally make sense, give the video a like on YouTube and subscribe for more problem breakdowns like this one. For deeper write-ups and series updates, subscribe to the Substack. Drop a comment with your own approach, or share this with someone prepping for interviews.

Quick Review

When costing a window to match the most-common value, what's the tell?

Window-equalization cost problems: pick the mode as target, cost = total elements minus count of the mode within the window.

Time complexity for sliding-window-with-frequency-cost on array of length n?

O(n log n): each window update costs O(log n) via a sorted structure (Fenwick/ordered multiset) tracking frequencies.

Space complexity for the frequency-tracking sliding window approach?

O(n): one Fenwick tree / counter indexed over distinct values, plus a frequency map, independent of window count.

Common bug when recomputing window cost incrementally?

Forgetting to update the running mode when an element's count ties the current max after sliding — stale 'best' leads to wrong minimum cost.

Sliding-window mode-cost vs brute-force recount per window — when does each apply?

Brute force recounts O(n) per window (O(n^2) total); incremental frequency structure reuses prior counts for O(log n) per shift.

Interview follow-up: why can't you greedily pick target per-window independently of k?

Because k disjoint subarrays must be chosen jointly — overlapping candidate windows compete, so you need DP over best non-overlapping window costs.
